

Advance CC-2

Note: This is for reference purposes only when you are practicing for placements or doing revision for exams.

Loop Detection in a Linked List

Intuition:-

Loop Detection algorithm in a **singly linked list** using Floyd's Tortoise and Hare algorithm. This is a well-known algorithm for detecting cycles in a linked list, and it works by using two pointers, one moving slowly (one step at a time) and the other moving quickly (two steps at a time). If there is a cycle in the linked list, the fast pointer will eventually meet the slow pointer.

Key Points:

- **Node Class:**
 - A Node class is defined with two fields:
 - **data:** The integer data stored in the node.
 - **next:** A reference to the next node in the linked list.
 - The constructor of the Node class initializes the data and sets next to null.
- **Linked List Construction:**
 - The insert(int n) method adds a new node with data n at the end of the linked list. If the list is empty (i.e., head is null), it initializes the list with the new node as the head.
 - The method traverses the list to find the last node and appends the new node to it.
- **Creating a Loop in the Linked List:**
 - The createLoop(int a, int b) method creates a loop in the linked list. Here:
 - a is the data value of the node where the loop will point to.
 - b is the number of steps from the head node at which the loop will be created.

- It first locates the node with data a, then traverses b steps from the head node and sets the next pointer of the node at position b to point to the node with data a, creating a loop.

Loop Detection Using Floyd's Tortoise and Hare Algorithm:

Code:

```
import java.util.*;
class Node{
    int data;
    Node next;
    Node(int data){
        this.data=data;
        this.next=null;
    }
}
class Main{
    static Node head=null;
    static void insert(int n){
        Node newNode = new Node(n);
        if(head==null){
            head=newNode;
        }
        else{
            Node curr = head;
            while( curr.next != null ){
                curr = curr.next;
            }
            curr.next = newNode;
        }
    }
    static boolean createLoop(int a,int b){
        int count = 0;
        Node n1 = head;
        Node n2 = head;
```

```

while( n1 != null && n1.data != a){
    n1 = n1.next;
}
if(n1 == null)
    return false;
while(count < b && n2!=null){
    n2 = n2.next;
    count++;
}
n2.next = n1;
return true;
}
static boolean detectLoop(Node head){
    Node slow = head;
    Node fast = head;

    while(fast!=null && fast.next!=null){
        slow=slow.next;
        fast=fast.next.next;
        if(slow == fast){
            return true;
        }
    }
    return false;
}
public static void main (String[] args) {
    Scanner sc=new Scanner(System.in);
    int n=sc.nextInt();
    for(int i=0;i<n;i++){
        insert(sc.nextInt());
    }
    int a=sc.nextInt();
    int b= n-1;
    createLoop(a,b);
    boolean d = detectLoop(head);
    if(d){
        System.out.println("Detected");
    }
    else{

```

```
        System.out.println("Not Detected");
    }
}
```

Time Complexity : $O(N)$

Space Complexity : $O(\text{constant})$

Online Platform Reference Links:

1. [Find the first node of the Loop in Linked List](#)
2. [Count Loop Length in Linked List](#)
3. [Code Force - Floyd's Loop Detection](#)
4. [Hacker Earth - Detect and Remove Loop in Linked List](#)

***If you are doing a few more problem statements, that is appreciable.**

Youtube Reference - [DSA - Linked List Reference](#) & [Placement Reference](#)

1. [Detect Loop](#)
2. [Find the starting node of the loop](#)
3. [Find the Length of the Loop in Linked List](#)

Sort the Bitonic Double Linked List

Intuition:-

Given a bitonic doubly linked list. The task is to sort the given linked list. A bitonic doubly linked list is a doubly linked list that is **first increasing and then decreasing**. A **strictly increasing** or a **strictly decreasing** list is also a bitonic doubly linked list.

Key Points:

- **Two Pointers:**
 - first: Start at the head of the list (beginning of the increasing part).
 - last: Starts at the tail of the list (end of the decreasing part).
- **Merging Logic:**
 - Compare first.data and last.data:
 - Append the smaller value to the result list.
 - Move the respective pointer (first forward or last backward).
- **In-Place Reorganization:**
 - Adjust the next and prev pointers to build the sorted list.
 - No extra space is required.
- **Handles All Cases:**
 - Works for fully increasing, fully decreasing, and edge cases like single-node or empty lists.

Code:

```
import java.util.Scanner;
class Main{
    static node head=null;
    static class node{
        int data;
        node next;
        node prev;
        node(int n){
            data=n;
            next=null;
            prev=null;
        }
    }
}
static void insert(int n){
    node newnode = new node(n);
    if(head==null) head=newnode;
    else{
```

```

        node cur=head;
        while(cur.next!=null) cur=cur.next;
        cur.next=newnode;
        newnode.prev=cur;
    }
}
static void bit(){
    node first=head;
    node last=head;
    node res=null;
    node resend=null;

    while(last.next!=null) last=last.next;
    while(first!=last){
        if(first.data<=last.data){
            if(res==null){
                res=resend=first;
                first=first.next;
            }
            else{
                node cur=first.next;
                resend.next=first;
                first.prev=resend;
                cur.prev=null;
                first=cur;
                resend=resend.next;
            }
        }
        else{
            if(res==null){
                res=resend=last;
                last=last.prev;
            }
            else{
                node cur=last.prev;
                resend.next=last;
                last.prev=resend;
                cur.next=null;
                last=cur;
            }
        }
    }
}

```

```

        resend=resend.next;
    }
}
resend.next=first;
first.prev=resend;
head=res;
}
static void display(){
    node cur=head;
    while(cur!=null){
        System.out.print(cur.data+"-->");
        cur=cur.next;
    }
    System.out.print("null");
}
public static void main(String ar[]){
    Scanner sw = new Scanner(System.in);
    int n=sw.nextInt();
    for(int i=0;i<n;i++){
        insert(sw.nextInt());
    }
    display();
    bit();
    System.out.println();
    display();
}
}

```

Time Complexity : $O(N)$

Space Complexity : $O(\text{constant})$

Online Platform Reference Links:

1. [Sort the Bitonic Double Linked Lis](#)

***If you are doing a few more problem statements, that is appreciable.**

Youtube Reference - [DSA - Linked List Reference](#) & [Placement Reference](#)

Segregate Even and Odd nodes in a Linked List

Intuition:-

Given a Linked List of integers, The task is to modify the linked list such that all even numbers appear before all the odd numbers in the modified linked list. Also, preserve the order of even and odd numbers. The task is to segregate the nodes of a singly linked list into two groups:

- **Even nodes:** Nodes with even values.
- **Odd nodes:** Nodes with odd values.

The resulting list will first contain all even nodes, followed by all odd nodes, while preserving the original relative order within each group.

Key Points:

- **Two Separate Sub-Lists:**
 - Create two sub-lists:
 - One for even nodes (es and ee: even start and even end).
 - One for odd nodes (os and oe: odd start and odd end).
 - Traverse the original list, appending nodes to the appropriate sub-list based on their values.
- **Combine the Two Sub-Lists:**
 - If the even list is non-empty:
 - Make the even list the head of the resulting list.
 - Link the last node of the even list (ee) to the head of the odd list (os).
 - If the even list is empty:
 - Make the odd list the head.
- **Preserve Original Order:**
 - Nodes are appended to the sub-lists in the order they appear in the original list, ensuring that the relative order within the even and odd groups is maintained.
- **Handles Edge Cases:**
 - Fully even or fully odd lists.
 - Empty list (no nodes).

Code:

```
import java.util.Scanner;
class Main{
    static node head=null;
    static class node{
        int data;
        node next;
        node(int n){
            data=n;
            next=null;
        }
    }
    static void insert(int n){
        node newnode = new node(n);
        if(head==null) head=newnode;
        else{
            node cur=head;
            while(cur.next!=null) cur=cur.next;
            cur.next=newnode;
        }
    }
    static void seg(){
        node es=null;
        node ee=null;
        node os=null;
        node oe=null;
        node cur=head;
        while(cur!=null){
            if(cur.data%2==0){
                if(es==null) es=ee=cur;
                else ee=ee.next=cur;
            }
            else{
                if(os==null) os=oe=cur;
                else oe=oe.next=cur;
            }
        }
    }
}
```

```

        cur=cur.next;
    }
    if(es==null) head=os;
    else{
        head=es;
        ee.next=os;
        oe.next=null;
    }
}

static void display(){
    node cur=head;
    while(cur!=null){
        System.out.print(cur.data+"-->");
        cur=cur.next;
    }
    System.out.print("null");
}

public static void main(String ar[]){
    Scanner sw = new Scanner(System.in);
    int n=sw.nextInt();
    for(int i=0;i<n;i++){
        insert(sw.nextInt());
    }
    display();
    seg();
    System.out.println();
    display();

}
}

```

Time Complexity : $O(N)$

Space Complexity : $O(\text{constant})$

Online Platform Reference Links:

1. [Segregate Even and Odd nodes in a Linked List - Code Chef](#)
2. [Swap nodes in Pairs - Code Chef](#)

3. [Reverse a Linked List - Code Chef](#)
4. [Reversed Linked List - Hacker Earth](#)

***If you are doing a few more problem statements, that is appreciable.**

Youtube Reference - [DSA - Linked List Reference](#) & [Placement Reference](#)

1. [Segregate Even and Odd Nodes in a Linked List](#)

Merge Sort for Double Linked List

Intuition:-

Given a doubly linked list, The task is to sort the doubly linked list in non-decreasing order using merge sort.. Also, preserve the order of even and odd numbers. The goal is to sort a **doubly linked list** (DLL) using the merge sort algorithm. Merge sort is a divide-and-conquer algorithm that splits the list into smaller parts, sorts them individually, and merges them back to create a fully sorted list. The doubly linked list's ability to traverse both forward and backward simplifies the merging process.

Key Points:

- **Divide and Conquer:**
 - Split the linked list into two halves using the **fast and slow pointer approach**.
 - Recursively sort each half.
- **Merge Two Sorted Lists:**
 - Combine two sorted sublists into a single sorted list.
 - Use the prev and next pointers to maintain the doubly linked structure.
- **Edge Cases:**
 - Empty list or single-node list: Return the list as it is already sorted.
 - Handle merging of sublists where one list is empty.

- **Preserve Node Links:**

- Ensure the prev and next pointers are correctly updated during the merge process to maintain the integrity of the doubly linked list.

Code:

```
import java.util.Scanner;
class Main{
    static node head=null;
    static class node{
        int data;
        node next;
        node prev;
        node(int n){
            data=n;
            next=null;
            prev=null;
        }
    }
    static void insert(int n){
        node newnode = new node(n);
        if(head==null) head=newnode;
        else{
            node cur=head;
            while(cur.next!=null) cur=cur.next;
            cur.next=newnode;
            newnode.prev=cur;
        }
    }
    static node sort(node first){
        if(first==null||first.next==null) return first;
        node second=split(first);
        first=sort(first);
        second=sort(second);
        return merge(first,second);
    }
}
```

```

static node split(node first){
    node fast=first;
    node slow=first;
    while(fast.next!=null&&fast.next.next!=null){
        fast=fast.next.next;
        slow=slow.next;
    }
    node temp=slow.next;
    slow.next=null;
    return temp;
}
static node merge(node first,node second){
    if(first==null) return second;
    if(second==null) return first;
    if(first.data<=second.data){
        first.next=merge(first.next,second);
        first.next.prev=first;
        first.prev=null;
        return first;
    }
    else{
        second.next=merge(first,second.next);
        second.next.prev=second;
        second.prev=null;
        return second;
    }
}
static void display(){
    node cur=head;
    while(cur!=null){
        System.out.print(cur.data+"-->");
        cur=cur.next;
    }
    System.out.print("null");
}
public static void main(String ar[]){
    Scanner sw = new Scanner(System.in);
    int n=sw.nextInt();
    for(int i=0;i<n;i++){

```

```
        insert(sw.nextInt());
    }
    head=sort(head);
    display();
}
}
```

Time Complexity : $O(N * \log N)$

Space Complexity : $O(\text{constant})$

Online Platform Reference Links:

***If you are doing a few more problem statements, that is appreciable.**

Youtube Reference - [DSA - Linked List Reference](#) & [Placement Reference](#)